NIRUPAM CHANGMAI

APPLIED AI / BACKEND ENGINEER

Guwahati, Assam, India | nirupamchangmai99@gmail.com | +91 9706409858
Portfolio | GitHub | Technical Writing | LinkedIn

PROFESSIONAL SUMMARY

Applied AI / Backend Engineer with production full-stack experience and a portfolio focused on the systems around modern LLM applications: agent runtimes, long-term memory, retrieval and reranking, document intelligence, local inference, API gateways, evaluation, streaming, authentication, quotas, and persistent state. Builds primarily with Python and FastAPI, while retaining strong TypeScript/Next.js product skills. Experience spans production APIs, AI integrations, relational databases, testing, and end-to-end product delivery.

TECHNICAL SKILLS

AI / LLM Systems: LLMs, AI agents, RAG, hybrid retrieval, reranking, embeddings, semantic search, vector databases, evaluation, provenance, local inference, prompt/structured-output workflows

Backend: Python, FastAPI, REST APIs, GraphQL, SQLAlchemy, Alembic, httpx, authentication/authorization, streaming/SSE, rate limits, quotas, background/runtime services

Data: PostgreSQL, SQLite, ChromaDB, BM25/lexical retrieval, persistent state, schema design, migrations

AI Infrastructure: llama.cpp, model/provider abstractions, model routing, admission control, queues, usage accounting, resumable execution, memory services

Document Intelligence: PyMuPDF, Tesseract OCR, page classification/routing, layout analysis, reading order, structure-aware chunking, canonical IR/provenance

Frontend / Product: TypeScript, JavaScript, React, Next.js, Node.js, Shopify/Liquid, WebSockets

Quality / Tools: pytest, Git, GitHub, API testing, regression testing, retrieval benchmarks, lint/typecheck/build verification, AWS RDS, S3

PROFESSIONAL EXPERIENCE

Software Developer | TanTech LLC
Jan 2026 – Present

Remote

- Built and optimized GraphQL and REST APIs across 3 production products, reducing data-fetch latency by 40% and improving backend reliability.
- Shipped AI workflows using OpenAI, Anthropic, and Gemini, including function calling, structured outputs, and prompt-driven automation that reduced manual workflow steps by 40%.
- Built shared platform and UI systems used across 3 products, reducing development time by 45% and improving application performance by 35%.
- Worked across backend APIs, AI integrations, frontend systems, databases, and production debugging in a full-stack product environment.

Software Engineer | XO Clothing
Jul 2025 – Dec 2025

Pune District, India

- Rebuilt and improved a Shopify storefront with a focus on responsive UX, performance, accessibility, and maintainable implementation.
- Built an internal production and inventory tracking platform using PostgreSQL, AWS RDS, and S3, replacing spreadsheet-driven operational workflows.
- Worked directly with stakeholders across product, content, and production to translate operational needs into software.

Freelance Software Engineer | Independent
Jan 2024 – Jul 2025

Remote

- Delivered 4 production full-stack applications using React, TypeScript, Node.js, Python, and PostgreSQL for independent clients.
- Integrated OpenAI and Gemini APIs into client-facing workflows, including structured AI-assisted features and application-specific automation.
- Built and customized e-commerce experiences, including Shopify/Liquid implementations, while owning delivery from requirements through deployment.

Senior Developer | Riyum Innovations Pvt Ltd
Mar 2024 – Jan 2025

Guwahati, Assam, India

- Built a real-time internal communication platform with React, Node.js, and WebSockets for 30+ concurrent users, replacing two separate tools and reducing response time by 35%.
- Developed an LMS using React, TypeScript, GraphQL, and PostgreSQL supporting 120+ active users with zero reported downtime during the stated production period.

- Introduced testing and QA practices that identified 15 critical issues before release and helped stabilize product delivery.
- Built a shared design system that reduced feature-development time by 45% across the team's applications.

SELECTED AI / BACKEND PROJECTS

Syn — Self-Hosted LLM Inference Gateway

Python · FastAPI · SQLAlchemy · Alembic · SQLite · httpx · pytest · llama.cpp · OpenAI-compatible APIs

- Built an OpenAI-compatible control plane for securely serving local LLMs through llama.cpp, separating stable client-facing APIs from backend/runtime details.
- Implemented API-key authentication, client/model access policies, bounded FIFO admission control, request queuing, SSE streaming with disconnect handling, per-minute rate limits, daily quotas, and durable usage accounting.
- Verified resource cleanup under client disconnects and designed the gateway for constrained single-GPU operation, where concurrency and failure isolation materially affect reliability.

Huginn — Durable Runtime for AI Agents

Python · FastAPI · SQLAlchemy · Alembic · SQLite · httpx · CLI · provider abstractions

- Built a model-agnostic agent runtime with persistent tasks, runs, steps, traces, tool orchestration, and resumable execution so interrupted workflows can continue instead of restarting.
- Designed an extensible tool registry with built-in tools, provider abstractions, CLI/API control paths, retries/failure recovery, approvals, tracing, and model-routing foundations.
- Integrated Munin through a memory-provider abstraction, keeping long-term memory lifecycle management separate from execution state.

Munin — Long-Term Memory Service for AI Agents

Python · FastAPI · SQLAlchemy · Alembic · SQLite · embeddings · semantic retrieval

- Built a standalone memory service that persists useful information outside an LLM's temporary context window and retrieves relevant memories across later runs.
- Implemented semantic search, memory admission, deduplication and reinforcement, contradiction handling, consolidation, importance/temporal relevance, filtering, and auditable memory decisions.
- Designed model- and agent-agnostic interfaces so memory can be consumed by runtimes such as Huginn without coupling the runtime to one storage or policy implementation.

Jung Archive — Evidence-First RAG & Document Intelligence System

Python · FastAPI · Next.js · TypeScript · ChromaDB · BM25 · sentence-transformers · cross-encoder reranking · PyMuPDF

- Built an inspectable RAG pipeline covering document intelligence, structure-aware chunking, dense retrieval, BM25 lexical search, Reciprocal Rank Fusion, cross-encoder reranking, evidence packaging, and source provenance.
- Created a 30-question retrieval benchmark; Hit@1 improved from 0.433 with dense retrieval to 0.767 using hybrid retrieval plus reranking.
- Built an inspector UI for document structure, chunks, retrieval traces, evaluation failures, PDF provenance, and an evidence-backed knowledge graph.

RagParser — Local-First Document Intelligence Engine for RAG

Python · PyMuPDF · Tesseract OCR · canonical IR · layout analysis · diagnostics

- Built page-level routing that classifies pages as NATIVE, OCR, HYBRID, EMPTY, SUSPICIOUS, or FAILED and selects the extraction path accordingly.
- Designed a canonical intermediate representation with provenance, diagnostics, reading order, layout analysis, and structural detection for headings, headers, footers, and page numbers.
- Separated document normalization from retrieval so downstream RAG systems receive structured, inspectable content rather than raw PDF extraction output.

Aletheia — Local LLM Benchmarking & Profiling Platform

Python · FastAPI · llama.cpp · local LLMs · benchmark APIs

- Built a local-first platform for running structured LLM benchmark experiments on consumer hardware rather than relying only on model-card claims.
- Captures runtime-oriented benchmark data such as latency, throughput/token-generation behavior, model configuration, and hardware constraints to support informed local-model selection.
- Designed around llama.cpp-backed local inference with API-driven benchmark runs and persistent experiment data.

Absurd RAG — Fully Local RAG Experiment on Albert Camus

Python · PyMuPDF · Tesseract · sentence-transformers · ChromaDB · llama.cpp

- Built a local RAG pipeline over five Albert Camus books, covering ingestion, page classification/OCR routing, structure-aware chunking, embeddings, retrieval, and local inference.
- Processed mixed native-text, scanned, OCR-required, empty, and structurally inconsistent PDF pages; generated 840 provenance-preserving chunks across the corpus.
- The document failures encountered in this project directly motivated the standalone RagParser architecture.

EDUCATION

Master of Computer Applications (MCA) | **Amity University Online**

In progress

Bachelor of Computer Applications (BCA) | **Completed**

[Add institution and graduation year before sending]

TECHNICAL WRITING

Substack: substack.com/@neerruu

- “Building the Door Between My Laptop and a Local LLM” — engineering notes on turning local inference into a usable gateway with auth, queues, streaming, quotas, routing, and observability.
- “I Tried to Build a RAG. The PDFs Had Other Plans.” — build log on document extraction failures, OCR, chunking, and the path from Absurd RAG to a dedicated parser.
- Published technical essays on local LLM setup/runtime behavior, replacing cloud-model dependencies, document parsing, RAG evaluation/provenance, and AI infrastructure.